システムアーキテクト試験 科目A-2（午前II）

直前見直しシート

計算公式・頻出用語・まぎらわしい区別を，本番直前に見返せる形にまとめました。

目次

1. 直前チェックリスト
2. 計算問題の公式と例題
3. デザインパターン・設計原則
4. 電子署名・PKI
5. 暗号
6. JIS X 0160 と共通フレーム
7. 契約形態
8. トランザクション
9. 信頼性設計の用語
10. DFD・UML・アジャイル
11. ネットワーク・メール・Web攻撃
12. 出題範囲の用語 総点検（55語）

1 直前チェックリスト

試験当日の朝にこのページだけ見ます。言えなかった項目は該当ページを開いてください。

- [ ] キャッシュの平均アクセス時間を **h·x＋(1−h)·y** で書ける（rがミス率なら (1−r)·x＋r·y）
- [ ] 平均値からヒット率を逆算できる（0.2h＋2.2(1−h)＝1.0 → h＝0.6）
- [ ] アムダールの法則 **1÷{(1−a)＋a/n}**，稼働率 **MTBF÷(MTBF＋MTTR)**，直列は積・並列は $1-(1-A)^2$
- [ ] サイクロマティック複雑度 **e−n＋2**（＝分岐数＋1）
- [ ] 「A又はB」の確率は $\mathbf{1-(1-P_A)(1-P_B)}$
- [ ] GoFは **生成・構造・振る舞い** の3分類。Observer＝通知，Strategy＝アルゴリズム差替え，Decorator＝機能追加，Singleton＝1つだけ
- [ ] SOLID：単一責任・開放閉鎖・リスコフ置換・インタフェース分離・依存性逆転
- [ ] 署名は**送信者の秘密鍵**で作り，**送信者の公開鍵**で検証。失効確認は CRL（リスト）と **OCSP**（オンライン）
- [ ] 共通鍵は AES（鍵 128/192/256・ブロック128）。RSA・楕円曲線・ElGamal は公開鍵
- [ ] 請負＝完成責任，準委任＝善管注意義務，派遣＝発注者が指揮命令。ラボ＝体制確保の代わりに発注量を約束
- [ ] 隔離性を上げる → 不整合な読込みも処理件数も**減る**
- [ ] フェールセーフ（安全側停止）／フェールソフト（縮退運転）／フォールトトレランス（冗長化で継続）／フールプルーフ（誤操作防止）
- [ ] DFDの4要素：源泉と吸収・プロセス・データフロー・データストア（実体・関連はE-R図）
- [ ] スクラム：レビュー＝製品を見る，レトロスペクティブ＝やり方を振り返る。INVEST＝よいユーザーストーリー
- [ ] L2：ブリッジ・L2スイッチ，L3：ルータ。587番＝SMTP-AUTH，SPF＝送信元IP，DKIM＝署名

出題分野と予想配分（試験要綱の中分類）

中分類（◎重点）	予想	直近5回	このシートの該当箇所
◎12 システム開発技術	**10**	9〜12	2章・3章・6章・10章・12章、再出題集
◎11 セキュリティ	**5**	4〜5	4章・5章・11章
○4 システム構成要素	**2**	1〜3	2章（計算）・9章
◎18 システム企画	**2**	1〜2	7章
○9 データベース	**1**	1〜3	8章
○13 ソフトウェア開発管理技術	**1**	0〜2	10章・12章
○3 コンピュータ構成要素	**1**	1〜1	2章（計算）
○17 システム戦略	**1**	0〜2	新規出題予想（AI・データ）
○10 ネットワーク	**1**	1〜1	11章
○7 ユーザーインタフェース	**1**	0〜1	12章・新規出題予想（UI）

システム開発技術（約10問）とセキュリティ（約5問）で合格ラインの15問に届く計算です。直前はこの2分野を最優先にしてください。各問の中分類はIPAが公表していないため，出題内容から判断した予想です。

2 計算問題の公式と例題

テーマ	公式	メモ
キャッシュ	平均＝$h \cdot T_c + (1-h) \cdot T_m$	h＝ヒット率。ミス率rなら $(1-r) \cdot x + r \cdot y$
2階層キャッシュ	$h_1T_1 + (1-h_1)\{h_2T_2 + (1-h_2)T_3\}$	外側から順に場合分け
ページフォールト	平均≒$T_m + p \cdot T_{pf}$	p＝発生率，単位をnsにそろえる
CPI・実行時間	平均CPI＝Σ(構成比×CPI) 時間＝命令数×平均CPI÷周波数	MIPS＝周波数(MHz)÷平均CPI
パイプライン	(段数＋命令数−1)×ピッチ	5段・100命令・1ns → 104ns
アムダールの法則	$1 \div \{(1-a) + a/n\}$	a＝並列化できる割合，n＝台数
稼働率	MTBF÷(MTBF＋MTTR)	直列：$A_1 \times A_2$　並列：$1-(1-A_1)(1-A_2)$
RAID容量	0：nC　1：nC/2　5：(n−1)C　6：(n−2)C　10：nC/2	C＝1台の容量
エラー埋込み法	総自然バグ＝検出自然×埋込み総数÷検出埋込み	残存＝総自然バグ−検出自然
サイクロマティック	e−n＋2	＝分岐ノード数＋1
リスク	ALE＝年間発生確率×1回の損失額	A又はB：$1-(1-P_A)(1-P_B)$
転送時間	データ量(bit)÷(回線速度×実効率)	1バイト＝8ビット
IT投資	回収期間＝投資額÷年間効果 NPV＝$\Sigma\, CF_t \div (1+r)^t$−初期投資	ROIは問題の定義に従う

例題1　キャッシュサーバ（2025年問22）

ヒット時0.2秒，ミス時2.2秒，現在の平均1.0秒。3年後にヒット率が半分になる。3年後の平均は？

1. $0.2h + 2.2(1-h) = 1.0$
2. $2.2 - 2.0h = 1.0 \rightarrow$ **$h = 0.6$**
3. 3年後 $h = 0.3$
4. $0.2 \times 0.3 + 2.2 \times 0.7 = 0.06 + 1.54 =$ **1.6秒**

例題2　命令キャッシュ（2022年問21）

キャッシュ（容量a，時間x），主記憶（容量b，時間y），キャッシュに存在しない確率r。平均アクセス時間は？

1. ある確率(1−r)のとき時間x，ない確率rのとき時間y
2. 平均＝**$(1-r) \cdot x + r \cdot y$**（容量a・bは関係ない）
3. 数値例：x＝10ns，y＝100ns，r＝0.1 → 9＋10＝19ns

「ミス時はキャッシュを見てから主記憶」で確認時間も数える指定なら $x + r \cdot y$ になる。問題文の条件を確認する。

例題3　ページフォールト（2019年問19）

主記憶300ns，100万回に1回ページフォールト，1回200ms。

1. 200ms＝2×10^{8}ns
2. 平均≒$300+10^{-6}\times2\times10^{8}=300+200=$**500ns**

例題4　全数検査の費用（2018年問10）

1. 検査なし：500×3%＝15個×200万＝3,000万円
2. 検査あり：検査500万＋出荷前修理10個×50万＝500万＋出荷後5個×200万＝1,000万 → 2,000万円
3. 差：**1,000万円**

例題5　リスク（2025年問17）

Q_A＝40%，Q_B＝10%。マルウェア対策 d_A＝75% のみ導入：

1. $P_A=0.4\times0.25=0.1$，$P_B=0.1$
2. $P_C=1-0.9\times0.9=$**0.19**（他の案より低い）

例題6　サイクロマティック複雑度（2025年問8）

エッジ7本（A→B，A→C，B→C，B→E，C→D，C→E，D→E），ノード5個 → 7−5＋2＝**4**

例題7　その他の数値例

- アムダール：a＝0.8，n＝4 → $1\div(0.2+0.2)=$**2.5倍**
- 稼働率：0.9の装置2台 → 直列0.81，並列 1−0.1×0.1＝0.99
- CPI：1GHz，100万命令，平均CPI 2.8 → $2.8\times10^{6}\div10^{9}=$**2.8ms**
- エラー埋込み：埋込み100，検出した埋込み50，検出した自然20 → 総自然40，残存20
- 転送：100MBを100Mbps・実効80% → 800Mbit÷80Mbps＝**10秒**

3 デザインパターン・設計原則

分類	パターン	一言で
生成	Singleton	インスタンスが1つだけであることを保証
	Factory Method	どのクラスを生成するかをサブクラスに任せる
	Abstract Factory	関連する部品群をまとめて生成する
	Builder	複雑なオブジェクトを段階的に組み立てる
	Prototype	既存インスタンスを複製して生成
構造	Adapter	インタフェースを変換して既存クラスを使う
	Bridge	機能の階層と実装の階層を分ける
	Composite	容器と中身を同一視して木構造を扱う
	Decorator	オブジェクトを包んで機能を動的に追加
	Facade	複雑なサブシステムに単純な窓口を用意
	Flyweight	同じオブジェクトを共有して資源を節約
	Proxy	代理オブジェクトがアクセスを仲介
振る舞い	Observer	状態変化を依存オブジェクトへ自動通知
	Strategy	アルゴリズムをカプセル化して差し替える
	State	状態ごとに振る舞いを切り替える
	Template Method	処理の骨組みは親，詳細はサブクラス
	Iterator	中身の実装を隠して順に走査する
	Command	要求をオブジェクトにする（取消し・履歴）
	Chain of Responsibility	処理できる者まで要求をたらい回し
	Mediator	仲介者がオブジェクト間のやり取りをまとめる
	Memento	状態を保存して元に戻せるようにする
	Visitor	データ構造と処理を分離する
	Interpreter	文法規則をクラスで表現する

SOLID原則

- **S** 単一責任：クラスの役割は1つ
- **O** 開放・閉鎖：拡張に開き，修正に閉じる
- **L** リスコフの置換：派生型は基本型と置き換え可能
- **I** インタフェース分離：利用者ごとにインタフェースを分ける
- **D** 依存性逆転：上位も下位も抽象に依存する

まぎらわしい区別

- GoF（生成・構造・振る舞い）とPOSA（Layers・Broker・MVC・Pipes and Filters・Microkernel）
- Strategy（委譲で差替え）／State（状態で切替え）／Template Method（継承）
- オーバーライド（同シグネチャで上書き）／オーバーロード（引数違いの多重定義）
- 結合度（弱→強）：データ＜スタンプ＜制御＜外部＜共通＜内容
- 強度（強→弱）：機能的＞情報的＞連絡的＞手順的＞時間的＞論理的＞暗合的

4 電子署名・PKI

送信者：本文のハッシュ値を**送信者の秘密鍵**で暗号化＝署名 → 受信者：署名を**送信者の公開鍵**で復号し，自分で計算したハッシュ値と比較 → 一致すれば「本人が作成」「改ざんなし」

機密性（暗号化）は**受信者の公開鍵**で暗号化し**受信者の秘密鍵**で復号。署名と逆になる点が頻出です。

用語	ポイント
認証局（CA）・ディジタル証明書	公開鍵とその持ち主をCAが署名で保証。形式はX.509
CRL	失効した証明書のシリアル番号一覧。定期的に取得
OCSP	失効状態をオンラインで問い合わせるプロトコル（OCSPレスポンダ）
コードサイニング証明書	ソフトウェアの発行元確認と改ざん検知
サーバ証明書（DV・OV・EV）	Webサイトの真正性。EVが最も審査が厳格
クライアント証明書	利用者・端末の認証
タイムスタンプ	その時刻に存在し，以後改ざんされていないことを証明
eシール	法人が発行元であることと改ざんなしを証明（電子版の社印）
TPM	鍵の生成・保管などを行う耐タンパ性のセキュリティチップ
HSM	鍵を安全に保管・処理する専用装置

5 暗号

	共通鍵暗号	公開鍵暗号
鍵	暗号化と復号で同じ鍵	公開鍵と秘密鍵のペア
代表例	AES，DES，3DES，Camellia，KCipher-2，ChaCha20	RSA（素因数分解），ElGamal・DH（離散対数），楕円曲線暗号
速度	速い	遅い
n人の鍵の数	n(n−1)/2	2n（n組）

AES

- 米国NISTが標準化
- ブロック長 **128ビット固定**
- 鍵長 **128／192／256ビット**（段数10／12／14）
- DES：鍵56ビット，ブロック64ビット

関連用語

- ハイブリッド暗号：共通鍵を公開鍵で配送し，本文は共通鍵で暗号化
- ハッシュ関数：SHA-256など。一方向性・衝突困難性
- 暗号利用モード：ECB，CBC，CTR，GCM（認証付き）
- CRYPTREC：電子政府推奨暗号リストを作成・評価
- 無線LAN：WPA2（AES-CCMP），WPA3（SAE）

6 JIS X 0160 と共通フレーム

JIS X 0160（ソフトウェアライフサイクルプロセス）は2022〜2024年に毎年出題され，毎回違うプロセスが問われます。共通フレーム2013は2022年以降出ていません。

プロセス群	プロセス
合意	取得，供給
組織のプロジェクトイネーブリング	ライフサイクルモデル管理，インフラストラクチャ管理，ポートフォリオ管理，人的資源管理，品質管理，知識管理
テクニカルマネジメント	プロジェクト計画，プロジェクトアセスメント及び制御，意思決定管理，リスク管理，構成管理，情報管理，測定，品質保証
テクニカル	ビジネス又はミッション分析，利害関係者ニーズ及び要求事項定義，システム／ソフトウェア要求事項定義，アーキテクチャ定義，設計定義，システム分析，実装，統合，検証，移行，妥当性確認，運用，保守，廃棄

よく問われる区別

- **検証**：要求事項どおりに作られたか（正しく作ったか）
- **妥当性確認**：利用者のニーズを満たすか（正しいものを作ったか）
- **移行**：運用環境へ移し，運用できる状態にする
- **廃棄**：システムを終了させ，データ・資産を処分する

V字の対応（共通フレーム）

- システム要件定義 ↔ システム適格性確認テスト
- システム方式設計 ↔ システム結合テスト
- ソフトウェア要件定義 ↔ ソフトウェア適格性確認テスト
- ソフトウェア方式設計 ↔ ソフトウェア結合テスト
- ソフトウェア詳細設計 ↔ ユニットテスト

JIS X 25010 品質特性

製品品質	副特性・定義のキーワード
機能適合性	ニーズを満たす機能を提供する度合い（完全性・正確性・適切性）
性能効率性	使用する資源の量に関係する性能（時間効率性・資源効率性・容量満足性）
互換性	共存性・相互運用性
使用性	目標を有効性・効率性・満足性をもって達成できる度合い
信頼性	指定条件下で機能を実行し続ける度合い（成熟性・可用性・障害許容性・回復性）
セキュリティ	機密性・インテグリティ・否認防止性・責任追跡性・真正性
保守性	モジュール性・再利用性・解析性・修正性・試験性
移植性	適応性・設置性・置換性

7 契約形態

	請負	準委任	労働者派遣
責任	仕事の完成（成果物）	業務の遂行（善管注意義務）	労働力の提供
指揮命令	ベンダー	ベンダー	**発注者（派遣先）**
契約不適合責任	あり	なし（成果完成型は報酬の扱いが異なる）	なし

契約方式	ポイント
定額（確定金額）	価格固定。コスト超過のリスクはベンダー
実費償還（コスト・プラス）	かかった費用を全額払い，報酬を上乗せ。リスクは発注者
タイム・アンド・マテリアル	単価×時間＋材料費
ラボ契約	スキル・人数の体制確保を求め，代わりに一定量の発注を約束
レベニューシェア	初期費用を抑え，システムが生む収益を分け合う
多段階契約	工程ごとに契約を分ける（要件定義は準委任，開発は請負など）

偽装請負：請負・準委任なのに発注者が作業者へ直接指示すること。知財：グラントバック＝改良技術の実施権をライセンサーに許諾，アサインバック＝権利を譲渡。

8 トランザクション

ACID特性

- **A** 原子性：全部実行か全部取消し
- **C** 一貫性：整合性制約を保つ
- **I** 隔離性：並行実行しても互いに影響しない
- **D** 耐久性：コミット結果は失われない

障害回復

- ロールバック：更新前ログで取消し（トランザクション障害）
- ロールフォワード：バックアップ＋更新後ログで復元（媒体障害）
- チェックポイント：回復の起点を記録
- WAL：ログを先に書く（ログ先行書込み）

隔離性水準	ダーティリード	ノンリピータブルリード	ファントム
READ UNCOMMITTED	起こる	起こる	起こる
READ COMMITTED	防ぐ	起こる	起こる
REPEATABLE READ	防ぐ	防ぐ	起こる
SERIALIZABLE	防ぐ	防ぐ	防ぐ

- 隔離性を上げる → 不整合な読込みは減り，同時実行性が下がって処理件数も**減る**
- 2相ロック：ロックを獲得する段階と解放する段階を分ける（直列化可能性を保証）
- 2相コミット：分散DBで「準備（全員OK？）→確定」の2段階でコミット
- デッドロックの検出：待ちグラフ（WFG）に閉路があればデッドロック

9 信頼性設計の用語

用語	考え方	例
フォールトアボイダンス	故障そのものを起こさない	高品質部品，十分なテスト
フォールトトレランス	故障しても機能を維持する	二重化（デュプレックス・デュアル）
フェールセーフ	故障時は安全側に倒す	信号機が赤になる
フェールソフト	機能を縮退して運転を続ける	一部停止でも残りで稼働
フールプルーフ	誤操作をしても危険にならない	ふたを閉めないと動かない
フォールトマスキング	故障の影響を外に見せない	多数決回路

分析手法

- FMEA：部品の故障モードから影響を評価（ボトムアップ）
- FTA：トップ事象から原因をAND/ORで木構造に（トップダウン）
- HAZOP：ガイドワードで設計意図からのずれを分析
- アシュアランスケース：主張と証拠で品質を論証（GSN）

ストレージ

- DAS：サーバ直結
- NAS：LAN上のファイルサーバ（ファイル単位）
- SAN：専用ネットワーク（FC・iSCSI，ブロック単位）
- シンプロビジョニング：実容量以上を論理的に割り当て，使った分だけ消費

10 DFD・UML・アジャイル

DFD

- 4要素：源泉と吸収（□）・プロセス（◯）・データフロー（→）・データストア（＝）
- 実体・関連はE-R図の要素
- 階層化では親子の入出力を一致させる（バランシング）
- プロセスには必ず入力と出力がある

UMLの使い分け

- ユースケース図：機能単位と外部との境界
- アクティビティ図：業務・処理の流れ（分岐・並行）
- シーケンス図：メッセージの時間順序（タイミング）
- ステートマシン図：状態と遷移
- クラス図：静的構造（可視性 + public，− private，# protected）

スクラム

- 役割：プロダクトオーナー，スクラムマスター，開発者
- イベント：スプリント，プランニング，デイリー，レビュー（製品），レトロスペクティブ（やり方）
- 作成物：プロダクトバックログ，スプリントバックログ，インクリメント
- KPT：Keep・Problem・Try

その他の頻出

- INVEST：独立・交渉可能・価値・見積り可能・小さい・テスト可能
- XP：ペアプログラミング（ドライバとナビゲータ），TDD，継続的インテグレーション
- 統一プロセス：ユースケース駆動・アーキテクチャ中心・反復的
- ペトリネット：並行事象の同期を表現
- SysML独自：要求図・パラメトリック図

11 ネットワーク・メール・Web攻撃

層	装置	見る情報
L1 物理層	リピータ，リピータハブ	電気信号
L2 データリンク層	ブリッジ，L2スイッチ	MACアドレス
L3 ネットワーク層	ルータ，L3スイッチ	IPアドレス
L4〜L7	ゲートウェイ	プロトコル変換

メールのセキュリティ

- OP25B：動的IPから外部25番への送信を遮断
- 587番：SMTP-AUTHで認証して送信（サブミッションポート）
- SPF：送信元IPが正規サーバかDNSで確認
- DKIM：電子署名をDNSの公開鍵で検証
- DMARC：認証失敗時の扱いを送信側が宣言
- ベイジアンフィルタ：単語の出現確率で迷惑メールを判定

攻撃と対策

- 格納型XSS：掲示板などに保存して閲覧者で発動／反射型：リンクを踏ませる
- SQLインジェクション：静的プレースホルダ
- CSRF：トークンで正規の画面からの要求か確認
- オープンリゾルバ：送信元偽装のDNS問合せで反射・増幅（DNSアンプ）
- オープンリレー：第三者中継を許すメールサーバ
- Smurf：送信元偽装のICMPを大量に返させる

プロトコル早見

- SSH：暗号化＋認証のリモートログイン（22番）
- OCSP：証明書の失効確認
- LDAP：ディレクトリアクセス，RADIUS：AAA認証
- SNMP：機器監視，CHAP：チャレンジレスポンス認証
- SIP（呼制御）・RTP（音声）：VoIP

2025年の新顔

- Keycloak：SSOを実現するOSS（OpenID Connect・SAML）
- オーケストレーション（中央の指揮役）とコレオグラフィ（自律分散）
- 協調フィルタリング：似た顧客の購入品を推薦
- ミューテーションテスト：変異を見抜けるかでテストの質を測る

12 出題範囲の用語 総点検（55語）

午前IIの出題範囲に挙がっている用語を，工程の順に1行ずつ確認できるようにしました。 追加 はこのシートに新しく加えた29語です。説明を読まずに意味が言えるかを試してから，答え合わせに使ってください。

要件定義・分析・モデリング	
システム要求事項分析	利害関係者の要求を，システムが満たすべき機能・性能・制約などの要求事項に変換し，検証できる形で定義する（JIS X 0160 のシステム要求事項定義）
CMMI 追加	開発プロセスの成熟度モデル。成熟度レベル：1 初期 → 2 管理された → 3 定義された → 4 定量的に管理された → 5 最適化している（V2.0 はレベル0 不完全を含む）
アジャイル開発プロセス	短い反復で動くソフトウェアを少しずつ提供する。スクラム・XP など。宣言の4価値：個人と対話，動くソフトウェア，顧客との協調，変化への対応（→10章）
システム要件分析（定義）	システムの機能要件・非機能要件を定義する。**システム適格性確認テスト**はここで定義した要件を基準に評価する（→6章）
データモデル	概念データモデル（E-R図：実体・関連・属性・多重度）→ 論理データモデル（関係モデル・正規化）→ 物理データモデル（DBMS固有の実装）。業務から作るトップダウン，既存帳票から作るボトムアップ
ソフトウェア要件定義	ソフトウェアに求める機能・性能・インタフェースなどを定義する。**ソフトウェア適格性確認テスト**と対応する
マイクロサービスアーキテクチャ 追加	小さく独立したサービスをAPIで連携させる。利点：サービスごとに変更・デプロイ・拡張しやすい。欠点：サービス間通信の負荷，データ整合性の確保が難しい
プロトタイプ 追加	試作品で要求を早期に確認する。**水平型**：画面など全体を浅く作る，**垂直型**：特定の機能を端から端まで深く作る。使い捨て型と進化型
DFD	源泉と吸収・プロセス・データフロー・データストアの4要素。階層化では親子の入出力を一致させる（→10章）
UML	オブジェクト指向のモデリング言語。構造図（クラス図など）と振る舞い図（ユースケース図，アクティビティ図，シーケンス図，ステートマシン図など）
クラス図	クラスと関連（集約・コンポジション・汎化・依存）・多重度などの静的構造。可視性：+ public，− private，# protected，~ package
シーケンス図	オブジェクト間のメッセージのやり取りを時間順（上から下）に表す。イベント処理のタイミング設計に向く
ステートマシン図	1つのオブジェクトの状態と，イベントによる状態遷移を表す
CRUDマトリックス 追加	機能（プロセス）とデータ（エンティティ）の関係を，生成（C）・参照（R）・更新（U）・削除（D）で表にしたもの。誰もCしないデータ，誰も参照しないデータなどの漏れ・矛盾を見つける
決定表（デシジョンテーブル） 追加	条件の組合せと実行する動作を表で整理する。条件記述部・条件指定部（Y/N）・動作記述部・動作指定部（X）で構成。複雑な条件の漏れ防止やテストケース設計に使う

BPMN 追加	業務プロセスを図示する国際標準の表記法。イベント（○），アクティビティ（角丸四角），ゲートウェイ（◇：分岐・合流），シーケンスフロー（→），プールとレーン（担当者・組織）
SysML	UMLを拡張したシステムズエンジニアリング用の言語。独自の図は**要求図とパラメトリック図**（制約条件を記述）
ペトリネット	プレース・トランジション・トークンで，並行して起こる事象の同期を表現する
設計	
システム方式設計	システム要件を，ハードウェア・ソフトウェア・手作業に割り当て，システムの最上位の構成を決める。**システム結合テスト**と対応
機能要件設計 追加	機能要件（業務として何をするか：処理・画面・帳票・データ）を満たす設計。非機能要件（性能・可用性・セキュリティ・運用など）の設計と区別する。IPA非機能要求グレードの6大項目：可用性，性能・拡張性，運用・保守性，移行性，セキュリティ，システム環境・エコロジー
組込みシステムのコデザイン 追加	ハードウェアとソフトウェアの協調設計。開発の早い段階で両者の機能分担をシミュレーションなどで評価し，並行して開発する
サーキットブレーカー 追加	障害が起きている呼出し先への呼出しを一時的に遮断して，障害の連鎖（カスケード障害）を防ぐ。マイクロサービスで使う
インプロセスデータベース 追加	DBエンジンをライブラリとしてアプリと同じプロセス・メモリ空間で動かす（SQLiteなど）。通信が不要で軽量。インメモリDB（全データをメモリに置く）と区別
サブルーチン 追加	引数の受渡し：値呼出し（呼出し元は変わらない），参照呼出し（アドレスを渡す），結果呼出し，名前呼出し。性質：再帰的（自分を呼べる），再入可能（同時に複数から呼べる），再使用可能（初期化して再利用できる），再配置可能（どの番地でも動く）
アシュアランスケース	主張と証拠で品質・安全性の達成を論証する文書。表記法はGSN（→9章）
品質特性	JIS X 25010 の製品品質8特性と利用時の品質5特性（→6章）
オブジェクト指向	カプセル化（データと操作をまとめ隠蔽），継承，汎化と特化（is-a），集約と分解（part-of），多態性（ポリモーフィズム），オーバーライドとオーバーロード
モジュール分割技法 追加	データの流れに着目：**STS分割**（源泉・変換・吸収），**TR分割**（トランザクションの種類ごと），**共通機能分割**。データ構造に着目：ジャクソン法，ワーニエ法
モジュール結合度	弱い順：データ ＜ スタンプ ＜ 制御 ＜ 外部 ＜ 共通 ＜ 内容。低いほどよい
モジュール強度	強い順：機能的 ＞ 情報的 ＞ 連絡的 ＞ 手順的 ＞ 時間的 ＞ 論理的 ＞ 暗合的。高いほどよい
アーキテクチャパターン	Layers，Pipes and Filters，MVC，Broker，Microkernel，Blackboard（POSA）。ほかにクライアントサーバ，イベント駆動，マイクロサービス
デザインパターン	GoFの23パターン（生成・構造・振る舞い）（→3章）

レビュー 追加	**インスペクション**：モデレータが主導する公式なレビュー（チェックリスト・記録・役割分担）。**ウォークスルー**：作成者が主催し，関係者とソースや仕様を追って誤りを探す。**ラウンドロビン**：参加者が順番に進行役を担う。**パスアラウンド**：成果物を回覧して指摘を集める
実装・テスト	
ユニフィケーション 追加	論理型言語（Prologなど）で，2つの項を一致させるように変数に値を代入する操作（単一化）。バックトラック（失敗したら別の候補を試す）と並ぶ推論の基本機能
MapReduce 追加	Mapでキーと値の組を生成し，同じキーごとにまとめてReduceで集計する，大量データの並列処理モデル（Hadoop）
ペアプログラミング	2人1組で，書く人（ドライバ）と見る人（ナビゲータ）を交代しながら開発する（XP）
アスペクト指向プログラミング 追加	ログ出力・トランザクション・認証のように多くのモジュールに横断する処理（横断的関心事）をアスペクトとして分離し，適用箇所を宣言して織り込む
コーディング規約 追加	命名・書式・禁止事項などを決め，品質と保守性をそろえる。組込みC言語ではMISRA C，セキュアコーディングではCERT C
アサーションチェック 追加	ある時点で成り立つべき条件（論理式）をプログラムに埋め込み，成り立たなければ検出する
流れ図 追加	JIS X 0121 の記号：端子（角丸），処理（長方形），判断（ひし形），入出力（平行四辺形），定義済み処理（両側に縦線の長方形），ループ端。手でトレースして実行順や結果を求める
テストの網羅性 追加	**命令網羅（C0）**：全命令を1回以上。**判定条件網羅（C1，分岐網羅）**：各判定の真偽を1回以上。**条件網羅**：各条件の真偽を1回以上（判定条件網羅を満たすとは限らない）。**判定条件／条件網羅**：両方を満たす。**複数条件網羅**：条件の全組合せ（最も強い）
ブラックボックステスト 追加	仕様からテストケースを作る。同値分割，境界値分析，デシジョンテーブル，原因結果グラフ，状態遷移テスト
実験計画法 追加	因子（条件）と水準（値）の組合せを統計的に選び，少ない試行回数で効果を調べる。テストでは直交表を使う
直交表 追加	どの2因子を取り出しても，水準の全組合せが同じ回数ずつ現れる表。全組合せを試さずにテストケースを減らせる。全ての2因子の組合せを最低1回含めるのはペアワイズ法
エラー埋込み法	総自然バグ数 ≒ 検出した自然バグ数 × 埋込み総数 ÷ 検出した埋込みバグ数（→2章）
リグレッションテスト 追加	修正や変更によって，変更していない部分に新たな不具合（デグレード）が生じていないかを確認するテスト（退行テスト）
全数検査	出荷前に全製品を検査する。検査費用と出荷後の故障費用の合計で効果を比べる（→2章の例題4）
探索的テスト技法 追加	事前に詳細な手順を決めず，テストの設計と実行を同時に進め，結果から学んで次のテストを決める

チューリングテスト 追加	判定者が会話の相手が人か機械かを見分けられるかで，機械の知性を判定する
移行・受入れ	
移行プロセス	新しいシステムを運用環境に移し，運用できる状態にする。移行方式：一斉移行，段階移行，並行移行（新旧を並行稼働），パイロット移行（一部部署で先行）。本番前に移行リハーサルを行う
ソフトウェア受入れテスト 追加	発注者（利用者）が，要件どおりに作られているかを受入れ基準に照らして確認し，受け入れるかを判断するテスト。運用テストを含むことが多い
カークパトリックモデル 追加	教育効果の4段階評価：レベル1 反応（満足度アンケート）→ 2 学習（理解度テスト）→ 3 行動（職場での実践）→ 4 結果（業績・ROI）
保守・廃棄	
ソフトウェア保守 追加	JIS X 0161（ISO/IEC 14764）の保守の種類：**是正保守**（見つかった問題の修正），**予防保守**（潜在的な問題を事前に修正），**適応保守**（環境の変化への対応），**完全化保守**（性能や保守性の改善）
廃棄プロセス	システムの稼働を終了させる。廃棄計画，データの消去・移行・保管，利用者への通知，契約やライセンスの終了，資産の処分
FMEA	構成要素の故障モードから影響度を評価する予防的な手法（ボトムアップ）。逆にトップ事象から原因を木構造にするのがFTA（→9章）